
FIT5196 Task 2 — Complete Interview Study Guide

What Task 2 Is Actually About

Before getting into any section, understand the goal precisely. The task is not to build the best possible linear regression model. The task is to prepare the data *for* a linear regression model. Two criteria drive every single decision in the notebook:

1. Features must be on the same scale — so no variable dominates the model simply because its numeric magnitude is large.
2. Features must have as linear a relationship as possible with the target — which requires fixing distributional skewness and heteroscedasticity.

Every transformation you apply, every method you reject, every comparison you make is evaluated against these two criteria. If the TA asks "why did you do X?", the answer always traces back to one or both of these.

The dataset is `suburb_info.xlsx` — 202 Melbourne suburbs. The five features are `number_of_houses`, `number_of_units`, `population`, `aus_born_perc`, `median_income`. The target is `median_house_price`. `suburb` is the index. `municipality` is a categorical column that is excluded from all transformation analysis.

Section 2: Setup and Data Loading

Why Three Columns Need Parsing

Three columns in the raw Excel file are stored as strings, not numbers, because they use formatting characters:

- `aus_born_perc` has a % suffix — e.g., "72%". Fix: `.str.rstrip('%').astype(float)`.
- `median_income` has a \$ prefix and comma-separated thousands — e.g., "\$1,234". Fix: `.str.replace('[$,]', '', regex=True).astype(float)`.
- `median_house_price` same format as `median_income`. Same fix.

If you don't parse these, pandas reads them as object dtype and all numeric operations fail. This is a data loading problem, not a transformation problem, and it must be resolved before anything else.

After parsing, shape is (202, 7), zero nulls, all numeric columns confirmed as correct dtypes.

Why Municipality Is Excluded

`municipality` is a categorical variable identifying the local government area of each suburb. You cannot scale or transform categorical data in any meaningful way for linear regression. The task explicitly asks for preparation of continuous numeric predictors. So `municipality` is excluded from the entire analysis — not dropped from the dataframe, just not included in `features`.

The five features list is defined explicitly:

```
features = ['number_of_houses', 'number_of_units', 'population', 'aus_born_perc',  
'median_income']  
target = 'median_house_price'
```

Section 3: EDA — This Is What Drives Every Decision

3.1 Numeric Summary — The Numbers You Need to Know

From the `describe()` output:

Column	Mean	Std	Min	Max
number_of_houses	4,155.77	3,400.86	283	23,338
number_of_units	2,480.41	3,132.68	60	24,380
population	18,659.31	9,604.19	170	54,005
aus_born_perc	68.37	10.76	36	88
median_income	1,387.94	317.84	633	2,352
median_house_price	1,218,816.83	471,680.74	507,200	2,875,800

The scale disparity is already visible here: `population` ranges to 54,005 while `aus_born_perc` ranges from 36 to 88. These can't go into a linear regression model on the same scale.

3.2 Skewness — The Core Diagnostic

Skewness values confirmed from the notebook:

Column	Raw Skewness
number_of_houses	2.1562
number_of_units	3.6302
population	1.0859
aus_born_perc	-0.5679
median_income	0.1725
median_house_price	1.0333

The interpretation rule the TA may ask about: Skewness beyond ± 1 indicates meaningful asymmetry that can harm linear regression assumptions. Beyond ± 0.5 is worth noting. Within ± 0.5 is approximately symmetric.

So from the numbers:

- **number_of_units** (3.63) and **number_of_houses** (2.16): heavily right-skewed. A small number of suburbs have disproportionately high values pulling the distribution right. These will definitely need non-linear transformation.
- **population** (1.09) and **median_house_price** (1.03): moderately right-skewed. Some correction needed.
- **aus_born_perc** (-0.57): mildly left-skewed, near-symmetric. Shape correction not strictly necessary.
- **median_income** (0.17): essentially symmetric. No shape correction needed at all.

Why does skewness harm linear regression? Right-skewed features produce heteroscedastic scatter plots — the variance fans out at higher values. You can see this in the scatter plots for **number_of_units** and **population** against **median_house_price**. Heteroscedasticity violates the constant-variance (homoscedasticity) assumption of OLS. It also tends to produce right-skewed residuals, violating the normality of residuals assumption. Both assumptions are in Section 3.6.

3.2.1 QQ Plots

QQ plots (quantile-quantile plots) map observed quantiles against theoretical normal quantiles. Points on the diagonal line = normal distribution. Systematic deviations reveal non-normality.

What the QQ plots showed for each column, consistent with skewness numbers:

- `number_of_units` and `number_of_houses`: strongest departures. Points deviate below the line in the lower tail and curve sharply above it in the upper tail — the classic heavy right-skew QQ signature.
- `population` and `median_house_price`: moderate upper-tail deviation.
- `aus_born_perc`: mild S-curve, lower-tail points below the line — mild left skew.
- `median_income`: tracks the reference line most closely of all — effectively symmetric.

Why include QQ plots at all if you already have skewness numbers? Because skewness is a summary statistic — a single number. QQ plots show *where* and *how* the departure from normality occurs. An S-shape tells you something different from a hockey-stick shape. The QQ plots are visual confirmation of the numeric skewness, not redundancy.

3.3 Scale Comparison — The Second Problem

Scale summary confirmed from the notebook:

Column	Range	Std
number_of_houses	23,055	3,400.86
number_of_units	24,320	3,132.68
population	53,835	9,604.19
aus_born_perc	52	10.76
median_income	1,719	317.84

`population` has a standard deviation 900 times larger than `aus_born_perc`. In the shared-axis boxplot, `aus_born_perc` is completely invisible — its entire range is compressed into a flat line at the bottom.

Why does scale disparity matter for linear regression? A linear model assigns a coefficient to each feature. If `population` has values in the tens of thousands while `aus_born_perc` has values around 36–88, the model is optimising coefficients on completely incompatible scales. A feature with large numeric magnitude will produce a large raw coefficient even if its actual predictive relevance is low. Scaling forces all features into comparable ranges so coefficient magnitudes reflect actual predictive contribution.

3.4 Correlation and Linearity with Target

Pearson correlations confirmed from the notebook:

Feature	r with median_house_price
median_income	0.7194
number_of_units	0.3439
aus_born_perc	0.3039
number_of_houses	-0.1045
population	-0.2872

median_income is the strongest predictor by far at $r = 0.72$. But the scatter plot shows variance increasing at higher income values — fan-shaped — which is heteroscedasticity. A high correlation coefficient doesn't mean the relationship is cleanly linear. The fan shape is the problem.

number_of_units ($r = 0.34$) and **aus_born_perc** ($r = 0.30$) have weak positive relationships. **population** ($r = -0.29$) and **number_of_houses** ($r = -0.10$) are negatively correlated, with **number_of_houses** showing virtually no linear signal.

Why is Pearson correlation used here? Because it measures linear association. A high Pearson r means the feature and target move together linearly. A low r can mean no relationship, or it can mean a non-linear relationship. The scatter plots distinguish between the two.

Critical observation about correlations across methods: The correlation values change negligibly across all transformation methods. **median_income** stays around 0.71–0.72 regardless of whether you use raw, log, sqrt, or Box-Cox. This tells you that transformations don't fundamentally change the predictive signal — they change distributional properties and therefore improve regression assumptions, not the raw correlation.

3.5 EDA Summary

Two problems identified and stated explicitly:

1. **Scale disparity** — **population** ranges to 54,005, **aus_born_perc** ranges to 88. All five features require scaling before modelling.
2. **Skewness** — **number_of_units** (3.63) and **number_of_houses** (2.16) heavily right-skewed; **population** (1.09) and **median_house_price** (1.03) moderately right-skewed. Right-skewed features produce heteroscedastic scatter — visible in the scatter plots. These need variance-stabilising transformation.

`aus_born_perc` and `median_income` are near-symmetric and primarily need scaling rather than shape correction.

3.6 OLS Assumptions — Why These Matter

Four assumptions of OLS stated in the notebook:

1. **Linearity** — each feature should have an approximately linear relationship with the target. The scatter plots already showed non-linear or fan-shaped relationships for several features.
2. **Homoscedasticity** — residuals should have constant variance across the range of predicted values. The fan-shaped scatter for `median_income` and `number_of_units` against the target indicates heteroscedasticity. Right-skewed distributions directly cause this.
3. **Normality of residuals** — residuals should be approximately normally distributed. Right-skewed features tend to produce right-skewed residuals. Fixing feature skewness before modelling helps.
4. **No multicollinearity** — features shouldn't be highly correlated with each other. This is a model selection concern, not a transformation concern. Noted but not addressed in this notebook.

Key statement from the notebook: Scaling methods (standardisation, min-max) address coefficient interpretability but do not improve any of assumptions 1–3. Non-linear transformations (log, square root, Box-Cox) can improve all three by reducing skewness and stabilising variance. This is the central argument for why linear transforms alone are insufficient.

Section 4: Transformation Analysis

The Structure of the Analysis

Each transformation is evaluated against the same two criteria: does it resolve scale disparity? Does it reduce skewness (improving linearity/homoscedasticity)? Each section also reports the Pearson correlation with the (appropriately transformed) target.

4.1 Standardisation

What it does: subtracts the mean, divides by the standard deviation. Output has mean = 0, std = 1 for every feature. Uses `StandardScaler()` from sklearn.

Results confirmed from notebook:

After standardisation: all means ≈ 0 ($\leq 1e-10$), all stds = 1.0025. Skewness values: `number_of_houses` 2.1562, `number_of_units` 3.6302, `population` 1.0859, `aus_born_perc` -0.5679, `median_income` 0.1725. These are **identical to the raw data** — not one digit changed.

Correlations with target: `median_income` 0.7194, `number_of_units` 0.3439, `aus_born_perc` 0.3039, `number_of_houses` -0.1045, `population` -0.2872. Again **identical to raw**.

Why is this expected? Standardisation is a linear transformation — $z = (x - \text{mean}) / \text{std}$. Linear transformations preserve the shape of the distribution. If the raw distribution is right-skewed, the standardised distribution is identically right-skewed, just rescaled. Pearson correlation is also invariant to linear transformations — multiplying or shifting both variables by constants doesn't change r .

Verdict: Resolves scale disparity. Does nothing for skewness or linearity assumptions. Insufficient on its own for the skewed columns.

4.2 Min-Max Normalisation

What it does: maps each column to [0, 1] by subtracting the minimum and dividing by the range. $(x - \min) / (\max - \min)$. Uses `MinMaxScaler()`.

Results confirmed from notebook:

After min-max: all values in [0.0, 1.0]. Skewness: `number_of_houses` 2.1562, `number_of_units` 3.6302, `population` 1.0859, `aus_born_perc` -0.5679, `median_income` 0.1725. **Identical to raw.** Correlations with target: **identical to raw.**

Why is this the same outcome as standardisation? Because min-max is also a linear transformation. $(x - \min) / (\max - \min)$ is just a linear rescaling — different formula, same mathematical class. Same consequences: shape unchanged, correlations unchanged.

Verdict: Resolves scale disparity. Does nothing for skewness. Insufficient on its own. Both standardisation and min-max are ruled out as sole solutions because they cannot correct skewness — a non-linear transformation is required.

Standardisation vs min-max — what's the practical difference? Min-max is sensitive to outliers because it uses the min and max. A single extreme value can compress the rest of the distribution. Standardisation is less sensitive because it uses mean and std, which are influenced by outliers but less so than the extremes. However, both fail equally at skewness correction, so this difference doesn't affect the Task 2 conclusion.

4.3 Log Transformation

What it does: applies `np.log(x)` to right-skewed columns. Compresses large values and stretches small ones, directly attacking right skewness. Applied only to

`number_of_houses`, `number_of_units`, `population` — the three right-skewed features. `aus_born_perc` and `median_income` are left unchanged (near-symmetric, no shape correction needed). Log is also applied to the target for correlation assessment.

Why not apply log to `aus_born_perc` and `median_income`? They're already near-symmetric. Applying log to a symmetric distribution doesn't improve anything and can introduce new skewness. Unnecessary transformation is not neutral — it can make things worse.

Results confirmed from notebook:

Column	Raw Skewness	Log Skewness
<code>number_of_houses</code>	2.1562	-0.4425
<code>number_of_units</code>	3.6302	-0.3994
<code>population</code>	1.0859	-2.6451
<code>aus_born_perc</code>	-0.5679	-0.5679 (unchanged)
<code>median_income</code>	0.1725	0.1725 (unchanged)

Correlations with `log(median_house_price)`: `median_income` 0.7198, `aus_born_perc` 0.3472, `number_of_units` 0.3097, `number_of_houses` -0.0788, `population` -0.1531.

The critical finding: log transformation is too aggressive for `population`. It overcorrects from +1.09 to -2.65 — you've replaced a right-skewed distribution with a severely left-skewed one. You've solved one problem and created another. `number_of_houses` and `number_of_units` are corrected well (-0.44 and -0.40), but the population problem rules out log as the sole transformation.

Why does log overcorrect population? Log's compression effect is strongest for distributions with a high proportion of small values. Population has a different distributional shape from `number_of_units` — its moderate initial skewness (1.09) means it doesn't need as aggressive a transformation. Log applies the same function everywhere regardless of each column's actual skewness level. A fixed exponent can't be optimal for all columns simultaneously.

Verdict: Improves skewness for `number_of_houses` and `number_of_units`. Overcorrects `population` (1.09 → -2.65). Not suitable as the sole method. Also doesn't resolve scale disparity — a separate scaling step would still be needed after log transformation.

4.4 Power (Square Root) Transformation

What it does: applies `np.sqrt(x)` to right-skewed columns. Equivalent to Box-Cox with $\lambda = 0.5$. Less aggressive than log (which is $\lambda = 0$). Applied to the same three columns as log. `aus_born_perc` and `median_income` unchanged.

Results confirmed from notebook:

Column	Raw Skewness	Sqrt Skewness
number_of_houses	2.1562	0.7625
number_of_units	3.6302	1.3239
population	1.0859	-0.0288
aus_born_perc	-0.5679	-0.5679 (unchanged)
median_income	0.1725	0.1725 (unchanged)

Correlations with `sqrt(median_house_price)`: `median_income` 0.7235, `number_of_units` 0.3499, `aus_born_perc` 0.3269, `number_of_houses` -0.0889, `population` -0.2538.

The finding: square root solves the population overcorrection problem — from 1.09 to -0.03, near-perfect symmetry. But it's now too gentle for the other two:

- `number_of_houses`: 2.16 → 0.76. Meaningful improvement but still noticeably right-skewed (above the ± 0.5 threshold for approximate symmetry).
- `number_of_units`: 3.63 → 1.32. Still well above the ± 1 threshold. Still meaningfully skewed.

The fundamental limitation of both log and square root: they are fixed-exponent transformations. Log is $\lambda = 0$, square root is $\lambda = 0.5$. A single fixed exponent cannot be simultaneously optimal for columns with different initial skewness levels.

`number_of_units` (skewness 3.63) needs something close to log. `population` (skewness 1.09) needs something gentler. No single fixed exponent satisfies both. Box-Cox solves this by optimising λ independently per column.

Verdict: Better than log for `population`. Worse than log for `number_of_houses` and `number_of_units`. Fixed exponent cannot be optimal for all columns. Same scale problem as log — no built-in standardisation.

4.5 Box-Cox Transformation

What it does: Box-Cox finds the optimal λ for each column independently — the value that best normalises that specific column's distribution. The family of transformations it covers:

- $\lambda = 0$: natural log
- $\lambda = 0.5$: square root
- $\lambda = 1$: identity (no transformation)
- $\lambda = 2$: square
- $\lambda < 0$: reciprocal-family

It's a generalisation of both log and power transformations. **Crucially requires strictly positive values.** All five features satisfy this (minimum values: `number_of_houses` 283, `number_of_units` 60, `population` 170, `aus_born_perc` 36, `median_income` 633).

`PowerTransformer(method='box-cox')` from `sklearn` is used. It also standardises the output to zero mean and unit variance by default. This means Box-Cox addresses both the skewness problem and the scale problem in a single step.

Optimal lambda values confirmed from notebook:

Column	λ
number_of_houses	0.1828
number_of_units	0.1269
population	0.5458
aus_born_perc	2.4607
median_income	0.7332

Interpreting the lambdas:

- `number_of_houses` ($\lambda = 0.18$) and `number_of_units` ($\lambda = 0.13$): very close to zero, confirming near-log behaviour. The log transformation was in the right direction for these columns; Box-Cox just does it more precisely.
- `population` ($\lambda = 0.55$): approximately square root ($\lambda = 0.5$). This explains why square root worked well for population — the optimal transformation is essentially square root. And it explains why log ($\lambda = 0$) overcorrected — it was too aggressive.
- `aus_born_perc` ($\lambda = 2.46$): above 1, meaning it benefits from an upward transformation (since it's slightly left-skewed, a power above 1 pushes the right tail out slightly). This also explains why log was not applied to `aus_born_perc` — log would make its left skew worse.
- `median_income` ($\lambda = 0.73$): between square root and identity. Near-symmetric to begin with, so the optimal λ is close to 1.

Results confirmed from notebook:

Column	Raw Skewness	Log Skewness	Box-Cox Skewness
number_of_houses	2.1562	-0.4425	-0.0064
number_of_units	3.6302	-0.3994	-0.0141
population	1.0859	-2.6451	0.1005
aus_born_perc	-0.5679	-0.5679	-0.1286
median_income	0.1725	0.1725	-0.0084

Every column is within ± 0.13 of zero. This is the only method that achieves near-perfect symmetry across all five features simultaneously.

Correlations with BoxCox(median_house_price): **median_income** 0.7151, **aus_born_perc** 0.3277, **number_of_units** 0.3197, **number_of_houses** -0.0870, **population** -0.2595.

Scale after Box-Cox: means ≈ 0 , stds = 1.0025 for all features. Confirmed in the final summary cell.

Section 5: Comparison and Evaluation

5.1 The Consolidated Skewness Table — The Whole Picture at Once

Full table confirmed from notebook:

Column	Raw	Std	Min-Max	Log	Sqrt	Box-Cox
number_of_houses	2.1562	2.1562	2.1562	-0.4425	0.7625	-0.0064
number_of_units	3.6302	3.6302	3.6302	-0.3994	1.3239	-0.0141
population	1.0859	1.0859	1.0859	-2.6451	-0.0288	0.1005
aus_born_perc	-0.5679	-0.5679	-0.5679	-0.5679	-0.5679	-0.1286
median_income	0.1725	0.1725	0.1725	0.1725	0.1725	-0.0084

Notice: Standardisation = Min-Max = Raw for skewness. Always. Log and Sqrt were only applied to the three right-skewed columns (**aus_born_perc** and **median_income** entries

in log and sqrt columns are unchanged because they weren't transformed). Box-Cox is the only method that brings every column inside ± 0.13 .

The Consolidated Correlation Table — Why It Doesn't Change Much

Column	Raw	Std	Min-Max	Log	Sqrt	Box-Cox
number_of_houses	-0.1045	-0.1045	-0.1045	-0.0788	-0.0889	-0.0870
number_of_units	0.3439	0.3439	0.3439	0.3097	0.3499	0.3197
population	-0.2872	-0.2872	-0.2872	-0.1531	-0.2538	-0.2595
aus_born_perc	0.3039	0.3039	0.3039	0.3472	0.3269	0.3277
median_income	0.7194	0.7194	0.7194	0.7198	0.7235	0.7151

All methods produce essentially the same correlation values. The range for `median_income` across all methods is 0.7151 to 0.7235 — less than 1 percentage point variation. `number_of_units` ranges from 0.3097 to 0.3499.

What this tells you: transformations don't dramatically change the linear predictive signal. They change the distributional properties and therefore improve regression assumptions (linearity, homoscedasticity, normality of residuals). They don't fundamentally reorganise which features are predictive. The TA might ask: "if correlation doesn't improve much, why transform at all?" The answer is: correlation improvement is not the goal. The goal is satisfying OLS assumptions, and the residual diagnostic in Section 5.1 directly shows that happening.

Important note about the correlation comparison: the Log, Sqrt, and Box-Cox columns compare against their respective transformed targets (log target, sqrt target, Box-Cox target), not the raw target. This is appropriate — when you transform the target, you're looking at the feature-transformed_target relationship. Comparing against the raw target after transforming features would be mixing scales.

5.1 Regression Diagnostics — The Decisive Evidence

Two OLS models fitted:

- Raw features → raw target: $R^2 = 0.6364$
- Box-Cox features → Box-Cox target: $R^2 = 0.6786$

R^2 improves from 0.6364 to 0.6786 — about 4 percentage points. This is a modest but real improvement.

But R^2 is not the main point. The main point is what the residual plots show.

Raw data residuals: clear fan shape. Variance increases as predicted values grow — residuals spread from roughly $\pm 250,000$ at the low end to over $\pm 1,000,000$ at the high end. This is textbook heteroscedasticity. It directly violates the constant-variance assumption of OLS.

Box-Cox residuals: the fan shape is largely eliminated. Residuals are distributed more evenly around zero across the range of predicted values. The spread is visibly more uniform.

This is the critical insight: Box-Cox transformation improves the suitability of the data for linear regression primarily by stabilising variance (fixing heteroscedasticity), not by boosting predictive power. A model with homoscedastic residuals produces valid standard errors and valid hypothesis tests. A model with heteroscedastic residuals has unreliable inference even if R^2 looks reasonable.

Section 6: Final Recommendation

Box-Cox applied uniformly to all five features and the target. Confirmed from the notebook's recommendation table.

Per-column justifications:

number_of_houses ($\lambda = 0.18$): Heavily right-skewed (2.16). Log corrected to -0.44 ; Box-Cox achieves -0.006 . Near-perfect. Log was already in the right direction, Box-Cox just does it more precisely because $\lambda = 0.18$ is near-log but optimised. Box-Cox's built-in standardisation also removes the scale disparity.

number_of_units ($\lambda = 0.13$): Most severely skewed feature (3.63). Log reduced to -0.40 ; Box-Cox corrects to -0.014 . Near-zero λ confirms the log-like transformation was appropriate — Box-Cox just finds the exact optimal level. Square root was clearly insufficient ($3.63 \rightarrow 1.32$ still meaningfully skewed).

population ($\lambda = 0.55$): Moderately right-skewed (1.09). This is the column where the method choice matters most. Log overcorrected to -2.65 — a severe new problem. Box-Cox with $\lambda = 0.55$ (approximately square root) corrects to 0.10 . The λ value itself explains everything: the optimal transformation is close to square root, not log. Log was too aggressive. Box-Cox found the right level automatically.

aus_born_perc ($\lambda = 2.46$): Mildly left-skewed (-0.57) and near-symmetric. Shape correction not strictly necessary, but Box-Cox reduces skewness to -0.13 and standardises the scale simultaneously. Even for near-symmetric columns, Box-Cox's built-in standardisation makes it a sensible choice — one step instead of two (transform + scale). $\lambda = 2.46$ means the optimal transformation is a moderate power-up function, which makes sense for mild left skew.

median_income ($\lambda = 0.73$): Near-symmetric (0.17). No meaningful shape correction needed. Box-Cox brings skewness to -0.008 and standardises. Standardisation alone would have been sufficient here, but Box-Cox is applied for consistency across all features. $\lambda =$

0.73 is between square root and identity — close to 1, reflecting that almost no transformation is needed.

median_house_price / target (Box-Cox): Moderately right-skewed (1.03). The residual diagnostic in Section 5.1 showed clear heteroscedasticity (fan-shaped residuals) when the raw target was used. After Box-Cox transformation, the fan shape was largely eliminated. Transforming the target is warranted because the raw target's skewness was producing non-constant residual variance. Box-Cox achieves near-zero target skewness (0.0135 confirmed from the notebook). **Important trade-off:** transforming the target means model predictions come out on the Box-Cox scale. To get predictions in dollars, you need to apply the inverse Box-Cox function. The notebook states this explicitly.

Summary of what was ruled out and why:

- Standardisation: ruled out as sole method — resolves scale but cannot correct skewness.
- Min-Max: ruled out as sole method — same limitation as standardisation, linear transformation.
- Log: ruled out as sole method — overcorrects `population` (1.09 → -2.65). Good for highly skewed columns but not universally applicable.
- Square root: ruled out as sole method — undercorrects `number_of_units` (3.63 → 1.32) and `number_of_houses` (2.16 → 0.76). Better than log for `population` but can't be optimal everywhere simultaneously.

Box-Cox subsumes all of the above. It corrects skewness per-column with an optimal λ , and its sklearn implementation standardises the output by default, addressing both criteria in one step.

Section 7: Conclusion Summary

Five transformation methods evaluated: standardisation, min-max normalisation, log transformation, square root transformation, Box-Cox transformation.

EDA identified two problems: severe scale disparity (`population` 54,005 vs `aus_born_perc` 88) and right skewness in four of six columns (`number_of_units` 3.63, `number_of_houses` 2.16, `population` 1.09, `median_house_price` 1.03).

Linear transforms resolved scale disparity but left skewness unchanged. Non-linear transforms addressed skewness but fixed-exponent methods couldn't be optimal for all columns. Box-Cox resolved both problems simultaneously with per-column optimised λ .

Final recommendation: Box-Cox applied to all five features and the target. All features land within ± 0.13 skewness of zero. All features at mean ≈ 0 , std ≈ 1 . R^2 improves from 0.6364 to 0.6786. Residual heteroscedasticity largely eliminated.

Numbers to Have Immediately Ready

Raw skewness:

- number_of_houses: 2.1562
- number_of_units: 3.6302
- population: 1.0859
- aus_born_perc: -0.5679
- median_income: 0.1725
- median_house_price (target): 1.0333

Box-Cox skewness (final):

- number_of_houses: -0.0064
- number_of_units: -0.0141
- population: 0.1005
- aus_born_perc: -0.1286
- median_income: -0.0084
- median_house_price (target): 0.0135

Box-Cox lambdas:

- number_of_houses: 0.1828
- number_of_units: 0.1269
- population: 0.5458
- aus_born_perc: 2.4607
- median_income: 0.7332

Population skewness trajectory: 1.0859 (raw) → -2.6451 (log) → -0.0288 (sqrt) → 0.1005 (Box-Cox)

R² comparison: 0.6364 (raw) → 0.6786 (Box-Cox)

Pearson correlations (raw): median_income 0.7194, number_of_units 0.3439, aus_born_perc 0.3039, number_of_houses -0.1045, population -0.2872

Dataset: 202 suburbs, 7 columns, 0 nulls after parsing

Every Q&A Angle the TA Can Take

"Why is municipality excluded?" It's categorical — a local government area label. You can't scale or transform categorical data meaningfully for linear regression. The task focuses on continuous numeric predictors.

"Why did you need to parse aus_born_perc, median_income, and median_house_price?" They were stored as strings in the Excel file — [aus_born_perc](#)

had a % suffix, the income and price columns had \$ prefixes and comma separators. Pandas reads them as object dtype. You need to strip the formatting characters and cast to float before any numeric operation.

"Why do standardisation and min-max both fail to address skewness?" Both are linear transformations — they multiply by a constant and/or add a constant. Linear transformations shift and scale distributions but preserve their shape. The skewness of a linear transform of X equals the skewness of X . The proof is algebraic, but the result is visible in the notebook: skewness values after standardisation are digit-for-digit identical to the raw skewness.

"Why not just standardise and call it done?" Standardisation resolves scale disparity but leaves skewness intact. Skewed features produce heteroscedastic feature-target relationships — the fan-shaped scatter is still there after standardisation. This violates OLS's homoscedasticity assumption and produces unreliable inference. You need a non-linear transformation to fix the shape.

"Why is log applied only to three columns?" Log is a shape correction for right-skewed distributions. `aus_born_perc` (−0.57) and `median_income` (0.17) are near-symmetric — applying log to a symmetric distribution doesn't improve it and can introduce new skewness. The TA might probe: "what would happen if you applied log to `aus_born_perc`?" The answer: `aus_born_perc` has $\lambda = 2.46$, meaning its optimal transformation is a power greater than 1, which is the opposite direction from log ($\lambda = 0$). Log would make its skewness worse.

"Why does log overcorrect population?" Log applies $\lambda = 0$ regardless of each column's initial skewness. `population` has moderate initial skewness (1.09) — it doesn't need the most aggressive transformation. Log compresses large values so aggressively that it pushes the right tail past symmetry into the left tail (−2.65). The optimal λ for population is 0.55 (close to square root), not 0.

"Why is square root better than log for population but worse for the others?" Square root is $\lambda = 0.5$ — between log ($\lambda = 0$) and identity ($\lambda = 1$). For `population` whose optimal λ is 0.55, square root is close to optimal. For `number_of_units` whose optimal λ is 0.13, square root overcorrects less than log but still undercorrects — $\lambda = 0.5$ is too far from the optimal 0.13. Fixed exponents can't be simultaneously optimal for distributions with different skewness levels.

"What is Box-Cox actually doing mathematically?" For each column, it tries different values of λ and selects the one that maximises the log-likelihood of the transformed data under a normal distribution. The transformation formula is: $(x^\lambda - 1) / \lambda$ for $\lambda \neq 0$, and $\log(x)$ for $\lambda = 0$. As $\lambda \rightarrow 0$, this converges to the log transformation. At $\lambda = 0.5$, it's proportional to square root minus a constant. The optimisation is over the entire distribution, so it naturally finds the right level for each column.

"Why does Box-Cox require strictly positive values?" The transformation formula involves x^λ . For non-integer λ , this is undefined for negative x (raises domain error). All five features satisfy this — the minimum values are all well above zero.

"Why transform the target?" The residual diagnostic confirmed that the raw target produces fan-shaped (heteroscedastic) residuals. Heteroscedastic residuals violate OLS's constant-variance assumption, making standard errors and p-values unreliable. Transforming the target to reduce its skewness ($1.03 \rightarrow 0.0135$ after Box-Cox) stabilises residual variance. The R^2 also improves from 0.6364 to 0.6786.

"What's the trade-off of transforming the target?" Predictions from the model come out on the Box-Cox scale. To get predictions in dollars, you apply the inverse Box-Cox function — `inverse_boxcox(y_hat, lambda)`. This is mentioned explicitly in the notebook. If you forget to back-transform, your predictions are meaningless numbers on the Box-Cox scale.

"Why did R^2 only improve by 4 percentage points? Is Box-Cox even worth it?" The point of Box-Cox here is not to dramatically boost R^2 — it's to satisfy OLS assumptions. A model with proper homoscedastic residuals produces valid standard errors, valid confidence intervals, and valid hypothesis tests. A model with heteroscedastic residuals may have a similar R^2 but its inference is unreliable. The residual plot improvement is the main evidence, not the R^2 change.

"If correlations barely change across methods, why do you care about transformations?" Because Pearson correlation measures linear association. Transformations don't reorganise which features are predictive — `median_income` stays at ~ 0.72 across all methods. Transformations change distributional properties (skewness, variance stability) which affect whether the linear regression assumptions are satisfied. Correlation is a necessary but not sufficient condition for a clean linear regression relationship.

"Standardisation vs min-max — when would you actually prefer one over the other?" Min-max is sensitive to outliers because it pins to the minimum and maximum. If there's one extreme value, the entire distribution gets compressed. Standardisation uses mean and std, which are influenced by outliers but more robust. Min-max is preferred when you need a specific output range (e.g., $[0, 1]$ for neural network inputs). Standardisation is preferred when the distribution is roughly normal and you care about preserving relative distances. In this notebook, both fail equally at skewness correction, so the choice doesn't affect the Task 2 outcome.

"Why Box-Cox uniformly for all five features rather than mixing methods?" Consistency and parsimony. Box-Cox with sklearn's PowerTransformer is a single call that handles all five features simultaneously, finds the optimal λ per column, and standardises the output. Mixing methods (log for some, sqrt for others, standardise for the rest) would require separate steps per column and is strictly inferior — Box-Cox subsumes log ($\lambda \approx 0$) and sqrt ($\lambda \approx 0.5$) as special cases. When a general method covers all special cases and performs strictly better, there's no reason to use the restricted versions.

"What does the λ value tell you about what transformation Box-Cox is actually applying?" $\lambda = 0$: log. $\lambda = 0.5$: square root. $\lambda = 1$: identity (no transformation). $\lambda = 2$: square. $\lambda = 2.46$ for `aus_born_perc` means a power slightly beyond quadratic — appropriate for

mild left skew since it stretches the right tail slightly. $\lambda = 0.13$ for `number_of_units` means near-log, consistent with its heavy right skew needing aggressive compression.

"How did you identify heteroscedasticity from the scatter plots in the EDA?" The scatter of `median_income` vs `median_house_price` shows a fan shape — at low income values, house prices cluster tightly; at high income values, house prices spread widely. Same pattern for `number_of_units`. Fan shapes in feature-target scatter plots indicate that the conditional variance of the target increases with the feature — exactly the definition of heteroscedasticity. This directly motivates variance-stabilising transformation before modelling.